

Direct LiDAR-Inertial Odometry: Lightweight LIO with Continuous-Time Motion Correction

Kenny Chen¹, Ryan Nemiroff¹, and Brett T. Lopez²

Abstract—Aggressive motions from agile flights or traversing irregular terrain induce motion distortion in LiDAR scans that can degrade state estimation and mapping. Some methods exist to mitigate this effect, but they are still too simplistic or computationally costly for resource-constrained mobile robots. To this end, this paper presents Direct LiDAR-Inertial Odometry (DLIO), a lightweight LiDAR-inertial odometry algorithm with a new coarse-to-fine approach in constructing continuous-time trajectories for precise motion correction. The key to our method lies in the construction of a set of analytical equations which are parameterized solely by time, enabling fast and parallelizable point-wise deskewing. This method is feasible only because of the strong convergence properties in our nonlinear geometric observer, which provides provably correct state estimates for initializing the sensitive IMU integration step. Moreover, by simultaneously performing motion correction and prior generation, and by directly registering each scan to the map and bypassing scan-to-scan, DLIO's condensed architecture is nearly 20% more computationally efficient than the current state-of-the-art with a 12% increase in accuracy. We demonstrate DLIO's superior localization accuracy, map quality, and lower computational overhead as compared to four state-of-the-art algorithms through extensive tests using multiple public benchmark and self-collected datasets.

I. INTRODUCTION

Accurate real-time state estimation and mapping are necessary capabilities for mobile robots to perceive, plan, and navigate through unknown environments. LiDAR-based localization has recently become a viable option for many mobile platforms, such as drones, due to more compact and accurate sensors. As a result, researchers have developed several new LiDAR odometry (LO) and LiDAR-inertial odometry (LIO) algorithms which often outperform vision-based approaches due to the superior range and depth measurement accuracy of a LiDAR. However, there are still fundamental challenges in developing reliable and accurate LiDAR-centric algorithms [1], especially for robots that execute agile maneuvers or traverse uneven terrain. In particular, such aggressive movements can induce significant distortion in the point cloud which corrupts the scan-matching process, resulting in severe or catastrophic localization error and map deformation.

Existing algorithms which attempt to compensate for this effect may work well in structured environments for non-holonomic systems (e.g., autonomous driving), but their performance can degrade under irregular conditions due to simplistic motion models, loss in precision from discretization,

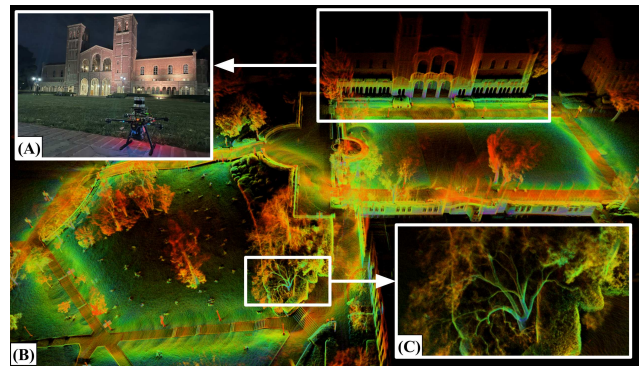


Fig. 1. **Real-time Localization and Dense Mapping.** DLIO generates detailed maps by reliably estimating robot pose, velocity, and sensor biases in real-time. (A) Our custom aerial vehicle next to UCLA's Royce Hall. (B) A bird's eye view of Royce Hall and its surroundings generated by DLIO. (C) A close-up of a tree, showcasing the fine detail that DLIO is able to capture in its output map. Color denotes intensity of point return.

and/or computational inefficiencies. For instance, works such as [2]–[4] assume constant velocity during scan acquisition which may work well for simple, predictable trajectories, but this quickly breaks down under significant acceleration. On the other hand, [5] and [6] use a back-propagation technique to mitigate distortion for each point, but their method may induce a loss in precision from accumulating integration error over time. More recently, continuous-time methods attempt to fit a smooth trajectory over a set of control points [7], [8] or augment scan-matching optimization with additional free variables [9], but such methods still hold strong assumptions on the trajectory (i.e., smooth movement) or may be too computationally costly for weight-limited platforms.

To this end, we present Direct LiDAR-Inertial Odometry (DLIO), a fast and reliable odometry algorithm that provides accurate localization and detailed 3D mapping (Fig. 1) with four main contributions. First, we propose a new coarse-to-fine technique for constructing continuous-time trajectories, in which a set of analytical equations with a constant jerk and angular acceleration motion model is derived for fast and parallelizable point-wise motion correction. Second, a novel condensed architecture is presented which combines motion correction and prior construction into one step and directly performs scan-to-map registration, significantly reducing overall computational overhead of the algorithm. Third, we leverage a new nonlinear geometric observer [10] that possesses strong performance guarantees—critical for achieving the first two contributions—in the pipeline to robustly generate accurate estimates of the robot's full state with minimal computational complexity. Finally, the efficacy of our approach is verified through extensive experimental results using multiple datasets against the state-of-the-art.

¹Kenny Chen and Ryan Nemiroff are with the Department of Electrical and Computer Engineering, University of California Los Angeles, Los Angeles, CA, USA. {kennyjchen, ryguyn}@ucla.edu

²Brett T. Lopez is with the Department of Mechanical and Aerospace Engineering, University of California Los Angeles, Los Angeles, CA, USA. btlopez@ucla.edu

All authors are with the Verifiable and Control-Theoretic Robotics Laboratory, University of California Los Angeles, Los Angeles, CA, USA.

DLIO is a lightweight LIO algorithm that generates robot state estimates and geometric maps through a unique architecture that contains two main components with three innovations (Fig. 2). The first is a fast scan-matcher which registers dense, motion-corrected point clouds onto the robot's map by performing alignment with an extracted local submap. Point-wise continuous-time integration in $\mathcal{W}$ ensures maximum image fidelity of the corrected cloud while simultaneously building in a prior for GICP optimization. In the second, a nonlinear geometric observer [10] updates the system's state with the first component's pose output to provide high-rate and provably correct estimates of pose, velocity, and sensor biases which converge globally. These estimates then initialize the next iteration of motion correction, scan-matching, and state update.

B. Notation

Let the point cloud for a single LiDAR sweep initiated at time t_k be denoted as $\mathcal{P}_k$ and indexed by k . The point cloud $\mathcal{P}_k$ is composed of points $p_k^n \in \mathbb{R}^3$ that are measured at a time Δt_k^n relative to the start of the scan and indexed by $n = 1, \dots, N$ where N is the total number of points in the scan. The world frame is denoted as $\mathcal{W}$ and the robot frame as $\mathcal{R}$ located at its center of gravity, with the convention that x points forward, y left, and z up. The IMU's coordinate system is denoted as $\mathcal{B}$ and the LiDAR's as $\mathcal{L}$, and the robot's state vector $\mathbf{X}_k$ at index k is defined as the tuple

$$\mathbf{X}_k = [\mathbf{p}_k^{\mathcal{W}}, \mathbf{q}_k^{\mathcal{W}}, \mathbf{v}_k^{\mathcal{W}}, \mathbf{b}_k^{\mathcal{a}}, \mathbf{b}_k^{\omega}]^{\top}, \quad (1)$$

where $\mathbf{p}^{\mathcal{W}} \in \mathbb{R}^3$ is the robot's position, $\mathbf{q}^{\mathcal{W}}$ is the orientation encoded by a four vector quaternion on $\mathbb{S}^3$ under Hamilton notation, $\mathbf{v}^{\mathcal{W}} \in \mathbb{R}^3$ is the robot's velocity, $\mathbf{b}^{\mathcal{a}} \in \mathbb{R}^3$ is the accelerometer's bias, and $\mathbf{b}^{\omega} \in \mathbb{R}^3$ is the gyroscope's bias. Measurements $\hat{\mathbf{a}}$ and $\hat{\omega}$ from an IMU are modeled as

$$\hat{\mathbf{a}}_i = (\mathbf{a}_i - \mathbf{g}) + \mathbf{b}_i^{\mathcal{a}} + \mathbf{n}_i^{\mathcal{a}}, \quad (2)$$

$$\hat{\omega}_i = \omega_i + \mathbf{b}_i^{\omega} + \mathbf{n}_i^{\omega}, \quad (3)$$

and indexed by $i = 1, \dots, M$ for M measurements between clock times t_{k-1} and t_k . With some abuse of notation, indices k and i occur at LiDAR and IMU rate, respectively, and will be written this way for simplicity unless otherwise stated. Raw sensor measurements $\mathbf{a}_i$ and ω_i contain bias $\mathbf{b}_i$ and white noise $\mathbf{n}_i$, and $\mathbf{g}$ is the rotated gravity vector. In this work, we address the following problem: given an accumulated point cloud $\mathcal{P}_k$ from a LiDAR and measurements $\mathbf{a}_i$ and ω_i sampled between each received scan by an IMU, estimate the robot's state $\hat{\mathbf{X}}_k^{\mathcal{W}}$ and the geometric map $\hat{\mathcal{M}}_k^{\mathcal{W}}$.

C. Preprocessing

The inputs to DLIO are a dense 3D point cloud collected by a modern 360° mechanical LiDAR, such as an Ouster or a Velodyne (10-20Hz), in addition to time-synchronized linear acceleration and angular velocity measurements from a 6-axis IMU at a much higher rate (100-500Hz). Prior

```

1 input:  $\hat{\mathbf{X}}_{k-1}^{\mathcal{W}}, \mathcal{P}_k^{\mathcal{L}}, \mathbf{a}_k^{\mathcal{B}}, \omega_k^{\mathcal{B}}$ ; output:  $\hat{\mathbf{X}}_k^{\mathcal{W}}, \hat{\mathcal{M}}_k^{\mathcal{W}}$ 
// LiDAR Callback Thread
2 while  $\mathcal{P}_k^{\mathcal{L}} \neq \emptyset$  do
// initialize points and transform to  $\mathcal{R}$ 
3  $\hat{\mathcal{P}}_k^{\mathcal{R}} \leftarrow \text{initializePointCloud}(\mathcal{P}_k^{\mathcal{L}})$  (Sec. III-C);
// continuous-time motion correction
4 for  $\hat{\mathbf{a}}_i^{\mathcal{R}}, \hat{\omega}_i^{\mathcal{R}}$  between  $t_{k-1}$  and  $t_k$  do
5  $\hat{\mathbf{p}}_i, \hat{\mathbf{v}}_i, \hat{\mathbf{q}}_i \leftarrow \text{discreteInt}(\hat{\mathbf{X}}_{k-1}^{\mathcal{W}}, \hat{\mathbf{a}}_{i-1}^{\mathcal{R}}, \hat{\omega}_{i-1}^{\mathcal{R}})$  (4);
6  $\hat{\mathbf{T}}_i^{\mathcal{W}} = [\hat{\mathbf{R}}(\hat{\mathbf{q}}_i) | \hat{\mathbf{p}}_i]$ ;
7 end
8 for  $p_k^n \in \hat{\mathcal{P}}_k^{\mathcal{R}}$  do
9  $\hat{\mathbf{T}}_n^{\mathcal{W}*} \leftarrow \text{continuousInt}(\hat{\mathbf{T}}_i^{\mathcal{W}*}, t_n)$  (5);
10  $\hat{p}_k^n = \hat{\mathbf{T}}_n^{\mathcal{W}*} \otimes p_k^n$ ;  $\hat{\mathcal{P}}_k^{\mathcal{W}}$ .append( $\hat{p}_k^n$ );
11 end
// scan-to-map registration
12  $\hat{\mathcal{S}}_k^{\mathcal{W}} \leftarrow \text{generateSubmap}(\hat{\mathcal{M}}_k^{\mathcal{W}})$  [20]
13  $\hat{\mathbf{T}}_k^{\mathcal{W}} \leftarrow \text{GICP}(\hat{\mathcal{P}}_k^{\mathcal{W}}, \hat{\mathcal{S}}_k^{\mathcal{W}})$  (6);
// geometric observer: state update
14  $\hat{\mathbf{X}}_k^{\mathcal{W}} \leftarrow \text{update}(\hat{\mathbf{T}}_k^{\mathcal{W}}, \Delta t_k^+)$  (Sec. III-F);
// update keyframe map
15 if  $\hat{\mathcal{P}}_k^{\mathcal{W}}$  is a keyframe then  $\hat{\mathcal{M}}_k^{\mathcal{W}} \leftarrow \hat{\mathcal{M}}_{k-1}^{\mathcal{W}} \oplus \hat{\mathcal{P}}_k^{\mathcal{W}}$ ;
16 return  $\hat{\mathbf{X}}_k^{\mathcal{W}}, \hat{\mathcal{M}}_k^{\mathcal{W}}$ 
17 end

// IMU Callback Thread
18 while  $\mathbf{a}_i^{\mathcal{B}} \neq \emptyset$  and  $\omega_i^{\mathcal{B}} \neq \emptyset$  do
// apply biases and transform to  $\mathcal{R}$ 
19  $\hat{\mathbf{a}}_i^{\mathcal{R}}, \hat{\omega}_i^{\mathcal{R}} \leftarrow \text{initializeImu}(\mathbf{a}_i^{\mathcal{B}}, \omega_i^{\mathcal{B}})$  (Sec. III-C);
// geometric observer: state propagation
20  $\hat{\mathbf{X}}_i^{\mathcal{W}} \leftarrow \text{propagate}(\hat{\mathbf{X}}_k^{\mathcal{W}}, \hat{\mathbf{a}}_i^{\mathcal{R}}, \hat{\omega}_i^{\mathcal{R}}, \Delta t_i^+)$  (Sec. III-F);
21 return  $\hat{\mathbf{X}}_i^{\mathcal{W}}$ 
22 end

```

to downstream tasks, all sensor data is transformed to be in $\mathcal{R}$ located at the robot's center of gravity via extrinsic calibration. For IMU, effects of displacing linear acceleration measurements on a rigid body must be considered if the sensor is not coincident with the center of gravity; this is done by considering all contributions of linear acceleration at $\mathcal{R}$ via the cross product between angular velocity and the offset of the IMU. To minimize information loss, we do not preprocess the point cloud except for a box filter of size 1m^3 around the origin to remove points that may be from the robot itself, and a light voxel filter for higher resolution clouds. This distinguishes our work from others that either attempt to detect features (e.g., corners, edges, or surfels) or heavily downsamples the cloud through a voxel filter.

D. Continuous-Time Motion Correction with Joint Prior

Point clouds from spinning LiDAR sensors suffer from motion distortion during movement due to the rotating laser array collecting points at different instances during a sweep. Rather than assuming simple motion (i.e., constant velocity) during sweep that may not accurately capture fine movement, we instead use a more accurate constant jerk and angular acceleration model to compute a unique transform for each point via a two-step coarse-to-fine propagation scheme. This strategy aims to minimize the errors that arise due to the

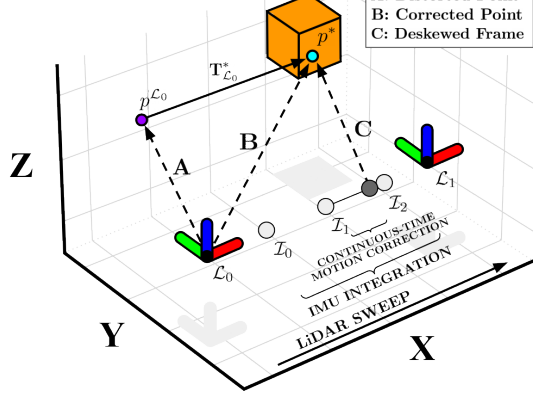


Fig. 3. **Coarse-to-Fine Point Cloud Deskewing.** A distorted point $p^{\mathcal{L}_0}$ (A) is deskewed through a two-step process which first integrates IMU measurements between scans, then solves for a unique transform in continuous-time (C) for the original point which deskews $p^{\mathcal{L}_0}$ to p^* (B).

sampling rate of the IMU and the time offset between IMU and LiDAR point measurements. Trajectory throughout a sweep is first coarsely constructed through numerical IMU integration [29], which is subsequently refined by solving a set of analytical continuous-time equations in $\mathcal{W}$ (Fig. 3).

Let t_k be the clock time of the received point cloud $\mathcal{P}_k^{\mathcal{R}}$ with N number of accumulated points within the time period, and let $t_k + \Delta t_k^n$ be the timestamp of a point p_k^n in the cloud. To approximate each point's location in $\mathcal{W}$, we first integrate IMU measurements between t_{k-1} and $t_k + \Delta t_k^N$ via

$$\begin{aligned}\hat{\mathbf{p}}_i &= \hat{\mathbf{p}}_{i-1} + \hat{\mathbf{v}}_{i-1}\Delta t_i + \frac{1}{2}\hat{\mathbf{R}}(\hat{\mathbf{q}}_{i-1})\hat{\mathbf{a}}_{i-1}\Delta t_i^2 + \frac{1}{6}\hat{\mathbf{J}}_i\Delta t_i^3, \\ \hat{\mathbf{v}}_i &= \hat{\mathbf{v}}_{i-1} + \hat{\mathbf{R}}(\hat{\mathbf{q}}_{i-1})\hat{\mathbf{a}}_{i-1}\Delta t_i, \\ \hat{\mathbf{q}}_i &= \hat{\mathbf{q}}_{i-1} + \frac{1}{2}(\hat{\mathbf{q}}_{i-1} \otimes \hat{\boldsymbol{\omega}}_{i-1})\Delta t_i + \frac{1}{4}(\hat{\mathbf{q}}_{i-1} \otimes \hat{\boldsymbol{\alpha}}_i)\Delta t_i^2,\end{aligned}\quad (4)$$

for $i = 1, \dots, M$ for M number of IMU measurements between two scans, where $\hat{\mathbf{J}}_i = \frac{1}{\Delta t_i}(\hat{\mathbf{R}}(\hat{\mathbf{q}}_i)\hat{\mathbf{a}}_i - \hat{\mathbf{R}}(\hat{\mathbf{q}}_{i-1})\hat{\mathbf{a}}_{i-1})$ and $\hat{\boldsymbol{\alpha}}_i = \frac{1}{\Delta t_i}(\hat{\boldsymbol{\omega}}_i - \hat{\boldsymbol{\omega}}_{i-1})$ are the estimated linear jerk and angular acceleration, respectively. The set of homogeneous transformations $\hat{\mathbf{T}}_i^{\mathcal{W}} \in \mathbb{SE}(3)$ that correspond to $\hat{\mathbf{p}}_i$ and $\hat{\mathbf{q}}_i$ then define the coarse, *discrete-time* trajectory during a sweep. Then, an analytical, *continuous-time* solution from the nearest preceding transformation to each point p_k^n recovers the point-specific deskewing transform $\hat{\mathbf{T}}_n^{\mathcal{W}*}$, such that

$$\begin{aligned}\hat{\mathbf{p}}^*(t) &= \hat{\mathbf{p}}_{i-1} + \hat{\mathbf{v}}_{i-1}t + \frac{1}{2}\hat{\mathbf{R}}(\hat{\mathbf{q}}_{i-1})\hat{\mathbf{a}}_{i-1}t^2 + \frac{1}{6}\hat{\mathbf{J}}_i t^3, \\ \hat{\mathbf{q}}^*(t) &= \hat{\mathbf{q}}_{i-1} + \frac{1}{2}(\hat{\mathbf{q}}_{i-1} \otimes \hat{\boldsymbol{\omega}}_{i-1})t + \frac{1}{4}(\hat{\mathbf{q}}_{i-1} \otimes \hat{\boldsymbol{\alpha}}_i)t^2,\end{aligned}\quad (5)$$

where $i-1$ and i correspond to the closest preceding and successive IMU measurements, respectively, t is the timestamp between point p_k^n and the closest preceding IMU, and $\hat{\mathbf{T}}_n^{\mathcal{W}*}$ is the transformation corresponding to $\hat{\mathbf{p}}^*$ and $\hat{\mathbf{q}}^*$ for p_k^n (Fig. 4). Note that (5) is parameterized only by t and therefore a transform can be queried for any desired time to construct a continuous-time trajectory.

The result of this two-step procedure is a motion-corrected point cloud that is also approximately aligned with the map in $\mathcal{W}$, which therefore inherently incorporates the optimization prior used for GICP (Sec. III-E). Importantly, (4) and (5)

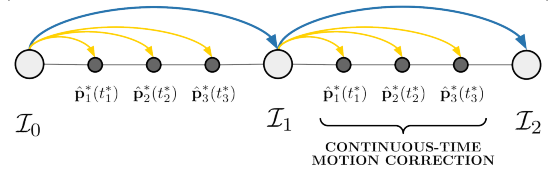


Fig. 4. **Continuous-Time Motion Correction.** For each point in a cloud, a unique transform is computed by solving a set of closed-form motion equations initialized at the closest preceding IMU measurement. This provides accurate and parallelizable continuous-time motion correction.

depend on the accuracy of $\hat{\mathbf{v}}_0^{\mathcal{W}}$, the initial estimate of velocity, $\mathbf{b}_k^a$ and $\mathbf{b}_k^{\omega}$, the estimated IMU biases, in addition to an accurate initial body orientation $\hat{\mathbf{q}}_0$ (to properly compensate for the gravity vector) at the time of motion correction. We therefore emphasize that, a key to the reliability of our approach is the *guaranteed global convergence* of these terms by leveraging DLIO's nonlinear geometric observer [10], provided that scan-matching returns an accurate solution.

E. Scan-to-Map Registration

By simultaneously correcting for motion distortion and incorporating the GICP optimization prior into the point cloud, DLIO can directly perform scan-to-map registration and bypass the scan-to-scan procedure required in previous methods. This registration is cast as a nonlinear optimization problem which minimizes the distance of corresponding points/planes between the current scan and an extracted submap. Let $\hat{\mathcal{P}}_k^{\mathcal{W}}$ be the corrected cloud in $\mathcal{W}$ and $\hat{\mathcal{S}}_k^{\mathcal{W}}$ be the extracted keyframe-based submap via [20]. Then, the objective of scan-to-map optimization is to find a transformation $\Delta\hat{\mathbf{T}}_k$ which better aligns the point cloud such that

$$\Delta\hat{\mathbf{T}}_k = \arg \min_{\Delta\mathbf{T}_k} \mathcal{E} \left(\Delta\mathbf{T}_k \hat{\mathcal{P}}_k^{\mathcal{W}}, \hat{\mathcal{S}}_k^{\mathcal{W}} \right), \quad (6)$$

where the GICP residual error $\mathcal{E}$ is defined as

$$\mathcal{E} \left(\Delta\mathbf{T}_k \hat{\mathcal{P}}_k^{\mathcal{W}}, \hat{\mathcal{S}}_k^{\mathcal{W}} \right) = \sum_{c \in \mathcal{C}} d_c^{\top} \left(C_{k,c}^{\mathcal{S}} + \Delta\mathbf{T}_k C_{k,c}^{\mathcal{P}} \Delta\mathbf{T}_k^{\top} \right)^{-1} d_c,$$

for a set of $\mathcal{C}$ corresponding points between $\hat{\mathcal{P}}_k^{\mathcal{W}}$ and $\hat{\mathcal{S}}_k^{\mathcal{W}}$ at timestep k , $d_c = \hat{s}_k^c - \Delta\mathbf{T}_k \hat{p}_k^c$, $\hat{p}_k^c \in \hat{\mathcal{P}}_k^{\mathcal{W}}$, $\hat{s}_k^c \in \hat{\mathcal{S}}_k^{\mathcal{W}}$, $\forall c \in \mathcal{C}$, and $C_{k,c}^{\mathcal{P}}$ and $C_{k,c}^{\mathcal{S}}$ are the estimated covariance matrices for point cloud $\hat{\mathcal{P}}_k^{\mathcal{W}}$ and submap $\hat{\mathcal{S}}_k^{\mathcal{W}}$, respectively. Then, following [13], this point-to-plane formulation is converted into a plane-to-plane optimization by regularizing covariance matrices $C_{k,c}^{\mathcal{P}}$ and $C_{k,c}^{\mathcal{S}}$ with $(1, 1, \epsilon)$ eigenvalues, where ϵ represents the low uncertainty in the surface normal direction. The resulting $\Delta\hat{\mathbf{T}}_k$ represents an optimal correction transform which better globally aligns the prior-transformed scan $\hat{\mathcal{P}}_k^{\mathcal{W}}$ to the submap $\hat{\mathcal{S}}_k^{\mathcal{W}}$, so that $\hat{\mathbf{T}}_k^{\mathcal{W}} = \Delta\hat{\mathbf{T}}_k \hat{\mathbf{T}}_M^{\mathcal{W}}$ (where $\hat{\mathbf{T}}_M^{\mathcal{W}}$ is the last point's IMU integration) is the globally-refined robot pose which is used for map construction and as the update signal for the nonlinear geometric observer.

F. Geometric Observer

The transformation $\hat{\mathbf{T}}_k^{\mathcal{W}}$ computed by scan-to-map alignment is fused with IMU measurements to generate a full state estimate $\hat{\mathbf{X}}_k$ via a novel hierarchical nonlinear geometric

Algorithm	Type	Absolute Trajectory Error (RMSE) [m]					Avg Comp. [ms]
		Short (1609.40m)	Long (3063.42m)	Quad (479.04m)	Dynamic (97.20m)	Park (695.68m)	
DLO [20]	LO	0.4633	0.4125	0.1059	0.1954	0.1846	48.10
CT-ICP [9]	LO	0.5552	0.5761	0.0981	0.1426	0.1802	412.27
LIO-SAM [4]	LIO	0.3957	0.4092	0.0950	0.0973	0.1761	179.33
FAST-LIO2 [6]	LIO	0.3775	0.3324	0.0879	0.0771	0.1483	42.86
DLIO (None)	LIO	0.4299	0.3988	0.1117	0.1959	0.1821	34.88
DLIO (Discrete)	LIO	0.3803	0.3629	0.0943	0.0798	0.1537	34.61
DLIO (Continuous)	LIO	0.3606	0.3268	0.0837	0.0612	0.1196	35.74

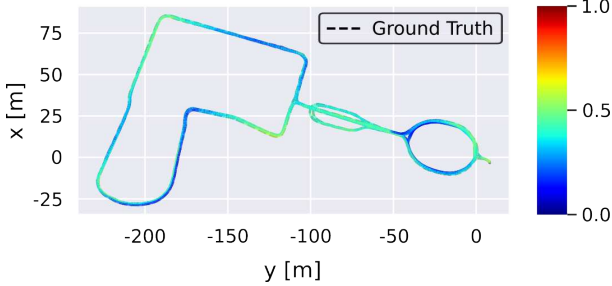


Fig. 5. **Trajectory of Long Experiment.** DLIO's generated trajectory for the Newer College - Long Experiment. Color indicates absolute pose error.

observer. A full analysis of the observer can be found in [10], but in summary, one can show that $\hat{\mathbf{X}}$ will globally converge to $\mathbf{X}$ in the deterministic setting with minimal computation. The proof utilizes contraction theory to first prove that the quaternion estimate converges exponentially to a region near the true quaternion. The orientation estimate then serves as an input to another contracting observer that estimates translation states. This architecture forms a contracting hierarchy that guarantees the estimates converge to their true values. This strong convergence result is the main advantage over other fusion schemes, e.g., filtering or pose graph optimization, which possess minimal convergence guarantees even in the most ideal setting. Additionally, the inherent smoothness of the observer's state estimate makes it suitable for control. The observer used in this work is a special case of the one in [10].

Let $\gamma_{\ell \in \{1, \dots, 5\}}$ be positive constants and Δt_k^+ be the time between GICP poses. If $\mathbf{q}_e := (q_e^o, \vec{q}_e) = \hat{\mathbf{q}}_i^* \otimes \hat{\mathbf{q}}_k$ and $\mathbf{p}_e = \hat{\mathbf{p}}_k - \hat{\mathbf{p}}_i$ (errors between propagated and measured poses) then the state correction takes the form

$$\begin{aligned}
 \hat{\mathbf{q}}_i &\leftarrow \hat{\mathbf{q}}_i + \Delta t_k^+ \gamma_1 \hat{\mathbf{q}}_i \otimes \begin{bmatrix} 1 - |q_e^o| \\ \text{sgn}(q_e^o) \vec{q}_e \end{bmatrix}, \\
 \hat{\mathbf{b}}_i^\omega &\leftarrow \hat{\mathbf{b}}_i^\omega - \Delta t_k^+ \gamma_2 q_e^o \vec{q}_e, \\
 \hat{\mathbf{p}}_i &\leftarrow \hat{\mathbf{p}}_i + \Delta t_k^+ \gamma_3 \mathbf{p}_e, \\
 \hat{\mathbf{v}}_i &\leftarrow \hat{\mathbf{v}}_i + \Delta t_k^+ \gamma_4 \mathbf{p}_e, \\
 \hat{\mathbf{b}}_i^a &\leftarrow \hat{\mathbf{b}}_i^a - \Delta t_k^+ \gamma_5 \hat{\mathbf{R}}(\hat{\mathbf{q}}_i)^\top \mathbf{p}_e.
 \end{aligned} \tag{7}$$

Note (7) is hierarchical as the attitude update (first two eqs.) is completely decoupled from the translation update (last three eqs.). Also, (7) is a fully nonlinear update which allows one to guarantee the state estimates are accurate enough to directly perform scan-to-map registration solely with an IMU prior without the need for scan-to-scan.

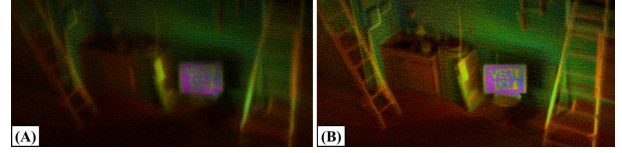


Fig. 6. **Deskewing Comparison.** Map generated from aggressive maneuvers without (A) and with (B) our motion correction method.

IV. RESULTS

DLIO was evaluated using the Newer College benchmark dataset [30] and data self-collected around the UCLA campus. We compare accuracy and efficiency against four state-of-the-algorithms, namely DLO [20], CT-ICP [9], LIO-SAM [4], and FAST-LIO2 [6]. Each algorithm employs a different degree and method of motion compensation, therefore creating an exhaustive comparison to the current state-of-the-art. Aside from extrinsics, default parameters at the time of writing for each algorithm were used in all experiments unless otherwise noted. Specifically, loop-closures were enabled for LIO-SAM and online extrinsics estimation disabled for FAST-LIO2 to provide the best results of each algorithm. For CT-ICP, voxelization was slightly increased and data playback was slowed down otherwise the algorithm would fail due to significant frame drops. All tests were conducted on a 16-core Intel i7-11800H CPU.

A. Ablation Study and Comparison of Motion Correction

To investigate the impact of our proposed motion correction scheme, we first conducted an ablation study with varying degrees of deskewing in DLIO using the Newer College dataset [30]. This study ranged from no motion correction (None), to correction using only nearest IMU integration via (4) (Discrete), and finally to full continuous-time motion correction via both (4) and (5) (Continuous) (Table I). Particularly of note is the Dynamic dataset, which contained highly aggressive motions with rotational speeds up to 3.5 rad/s. With no correction, error was the highest among all algorithms at 0.1959 RMSE. With partial correction, error significantly reduced due to scan-matching with more accurate and representative point clouds; however, using the full proposed scheme, we observed an error of only 0.0612 RMSE—the lowest among all tested algorithms. With similar trends for all other datasets, the superior tracking accuracy granted by better motion correction is clear: constructing a unique transform in continuous-time creates a more authentic point cloud than previous methods, which ultimately affects

Algorithm	Type	End-to-End Translational Error [m]				Avg. Comp. [ms]			
		A (652.66m)	B (526.58m)	C (551.38m)	D (530.75m)	A	B	C	D
DLO [20]	LO	0.0216	1.2932	0.0375	0.0178	20.40	20.77	21.18	21.62
CT-ICP [9]	LO	0.0387	0.0699	0.0966	0.0253	351.85	342.76	334.15	370.19
LIO-SAM [4]	LIO	0.0216	0.0692	0.0936	0.0249	33.21	29.14	39.04	48.94
FAST-LIO2 [6]	LIO	0.0454	0.0353	0.0363	0.0229	15.39	12.25	14.84	15.01
DLIO	LIO	0.0105	0.0233	0.0301	0.0082	10.45	8.37	8.66	10.96

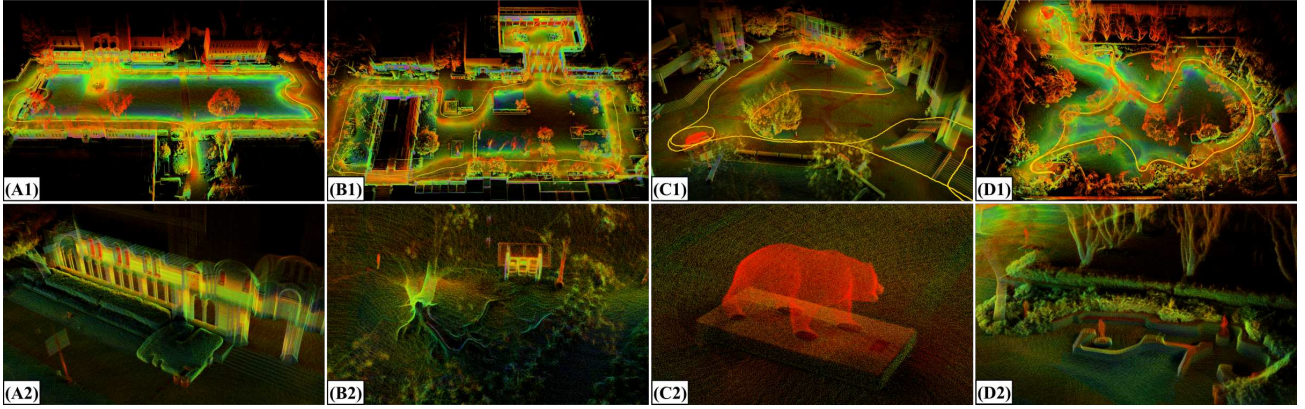


Fig. 7. **UCLA Campus.** Detailed maps of locations around UCLA in Los Angeles, CA generated by DLIO, including (A) Royce Hall in Dickson Court, (B) Court of Sciences, (C) Bruin Plaza, and (D) the Franklin D. Murphy Sculpture Garden, with both (1) a bird's eye view and (2) a close-up to demonstrate the level of fine detail DLIO can generate. The trajectory taken to generate these maps is shown in yellow in the first row.

scan-matching and therefore trajectory accuracy. Fig. 6 shows cases this empirically: DLIO can capture minute detail that is otherwise lost with simple or no motion correction.

B. Benchmark Results

1) *Newer College Dataset:* Trajectory accuracy and average per-scan time of all algorithms were also compared using the original Newer College benchmark dataset [30] via evo [31]. For these tests, we used data from the Ouster's IMU (100Hz) alongside LiDAR data (10Hz) to ensure accurate time synchronization between sensors. For certain Newer College datasets, the first 100 poses were excluded from computing FAST-LIO2's RMSE due to slippage at the start in order to provide a fair comparison. We also compared using the recent extension of the Newer College dataset [32] and observed similar results, but those results have been omitted due to space constraints. The results are shown in Table I, in which we observed DLIO to produce the lowest trajectory RMSE and lowest overall per-scan computational time (averaged across all five datasets) as compared to the state-of-the-art. Fig. 5 illustrates DLIO's low trajectory error compared to ground truth for the Newer College - Long Experiment dataset even after over three kilometers of travel.

2) *UCLA Campus Dataset:* We additionally collected four large-scale datasets at UCLA for additional comparison (Fig. 7). These datasets were gathered by hand-carrying our aerial platform (Fig. 1) over 2261.37m of total trajectory. Our sensor suite included an Ouster OS1 (10Hz, 32 channels recorded with a 512 horizontal resolution) and a 6-axis InvenSense MPU-6050 IMU located approximately 0.1m below it. We note here that this IMU can be purchased for approximately \$10, demonstrating that LIO algorithms need

not require high-grade IMU sensors that previous works have used. Note that a comparison of absolute trajectory error was not possible due to the absence of ground truth, so as is common practice, we compute end-to-end translational error as a proxy metric (Table II). In these experiments, DLIO outperformed all others across the board in both end-to-end translational error and per-scan efficiency. DLIO's resulting maps can capture fine detail in the environment which ultimately provides more intricate information cues for autonomous mobile robots such as terrain traversability.

V. CONCLUSION

This work presents Direct LiDAR-Inertial Odometry (DLIO), a highly reliable LIO algorithm that yields accurate state estimates and detailed maps in real-time for resource-constrained mobile robots. The key innovation that distinguishes DLIO from others is its fast and parallelizable coarse-to-fine approach in constructing continuous-time trajectories for point-wise motion correction. This approach is built into a simplified LIO architecture which performs motion correction and prior construction in one shot, in addition directly performing scan-to-map alignment for reduced computational overhead. This is all feasible due to our observer's strong convergence guarantees which reliably initializes pose, velocity, and biases for accurate IMU integration. Our experimental results demonstrate DLIO's improved localization accuracy, map clarity, and algorithmic efficiency as compared to the state-of-the-art, and future work includes closed-loop flight tests and adding loop closures.

Acknowledgements: The authors would like to thank Helene Levy and David Thorne for their help with data collection.

- [1] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.
- [2] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time," in *Robotics: Science and Systems*, vol. 2, no. 9, 2014, pp. 1–9.
- [3] T. Shan and B. Englot, "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018, pp. 4758–4765.
- [4] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 5135–5142.
- [5] W. Xu and F. Zhang, "Fast-lío: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.
- [6] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lío2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, 2022.
- [7] C. Park, P. Moghadam, S. Kim, A. Elfes, C. Fookes, and S. Sridharan, "Elastic lidar fusion: Dense map-centric continuous-time slam," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 1206–1213.
- [8] M. Ramezani, K. Khosoussi, G. Catt, P. Moghadam, J. Williams, P. Borges, F. Pauling, and N. Kottege, "Wildcat: Online continuous-time 3d lidar-inertial slam," *arXiv:2205.12595*, 2022.
- [9] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "Ct-icp: Real-time elastic lidar odometry with loop closure," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 5580–5586.
- [10] B. T. Lopez, "A contracting hierarchical observer for pose-inertial fusion," *arXiv:2303.02777*, 2023.
- [11] P. J. Besl and N. D. McKay, "Method for registration of 3-d shapes," in *Sensor fusion IV: control paradigms and data structures*, vol. 1611, 1992, pp. 586–606.
- [12] Y. Chen and G. Medioni, "Object modelling by registration of multiple range images," *Image and vision computing*, vol. 10, no. 3, pp. 145–155, 1992.
- [13] A. Segal, D. Hachnel, and S. Thrun, "Generalized-icp," in *Robotics: science and systems*, vol. 2, no. 4, 2009, p. 435.
- [14] T. Shan, B. Englot, C. Ratti, and D. Rus, "Lvi-sam: Tightly-coupled lidar-visual-inertial odometry via smoothing and mapping," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 5692–5698.
- [15] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li, "Mulls: Versatile lidar slam via multi-metric linear least square," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11 633–11 640.
- [16] T.-M. Nguyen, S. Yuan, M. Cao, L. Yang, T. H. Nguyen, and L. Xie, "Miliom: Tightly coupled multi-input lidar-inertia odometry and mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5573–5580, 2021.
- [17] H. Ye, Y. Chen, and M. Liu, "Tightly coupled 3d lidar inertial odometry and mapping," in *International Conference on Robotics and Automation*, 2019, pp. 3144–3150.
- [18] M. Palieri, B. Morrell, A. Thakur, K. Ebadi, J. Nash, A. Chatterjee, C. Kanellakis, L. Carlone, C. Guaragnella, and A.-a. Agha-Mohammadi, "Locus: A multi-sensor lidar-centric solution for high-precision odometry and 3d mapping in real-time," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, 2020.
- [19] A. Tagliabue, J. Tordesillas, X. Cai, A. Santamaria-Navarro, J. P. How, L. Carlone, and A.-a. Agha-mohammadi, "Lion: Lidar-inertial observability-aware navigator for vision-denied environments," in *International Symposium on Experimental Robotics*, 2020, pp. 380–390.
- [20] K. Chen, B. T. Lopez, A.-a. Agha-mohammadi, and A. Mehta, "Direct lidar odometry: Fast localization with dense point clouds," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2000–2007, 2022.
- [21] A. Reinke, M. Palieri, B. Morrell, Y. Chang, K. Ebadi, L. Carlone, and A.-A. Agha-Mohammadi, "Locus 2.0: Robust and computationally efficient lidar odometry for real-time 3d mapping," *IEEE Robotics and Automation Letters*, pp. 1–8, 2022.
- [22] J. Zhang, M. Kaess, and S. Singh, "On degeneracy of optimization-based state estimation problems," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 809–816.
- [23] in *2007 European Control Conference (ECC)*. IEEE, 2007.
- [24] J. F. Vasconcelos, C. Silvestre, and P. Oliveira, "A nonlinear gps/imu based observer for rigid body attitude and position estimation," in *2008 47th IEEE Conference on Decision and Control*. IEEE, 2008, pp. 1255–1260.
- [25] T. Renzler, M. Stolz, M. Schratte, and D. Watznig, "Increased accuracy for fast moving lidars: Correction of distorted point clouds," in *IEEE International Instrumentation and Measurement Technology Conference*, 2020.
- [26] S.-P. Deschênes, D. Baril, V. Kubelka, P. Giguere, and F. Pomerleau, "Lidar scan registration robust to extreme motions," in *Conference on Robots and Vision*, 2021.
- [27] D. Droschel and S. Behnke, "Efficient continuous-time slam for 3d lidar-based online mapping," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5000–5007.
- [28] C. Park, P. Moghadam, J. L. Williams, S. Kim, S. Sridharan, and C. Fookes, "Elasticity meets continuous-time: Map-centric dense 3d lidar slam," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 978–997, 2022.
- [29] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, 2016.
- [30] M. Ramezani, Y. Wang, M. Camurri, D. Wisth, M. Mattamala, and M. Fallon, "The newer college dataset: Handheld lidar, inertial and vision with ground truth," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 4353–4360.
- [31] M. Grupp, "evo: Python package for the evaluation of odometry and slam." <https://github.com/MichaelGrupp/evo>, 2017.
- [32] L. Zhang, M. Camurri, and M. Fallon, "Multi-camera lidar inertial extension to the newer college dataset," *arXiv:2112.08854*, 2021.